

Proyecto Final Integrador

Consigna, catálogo de dominios y rúbrica de evaluación — Semanas 13 a 16

1. Presentación y propósito

El proyecto final integrador es la instancia de cierre del curso. A lo largo de cuatro semanas, cada equipo aplica, sobre un problema de cierta envergadura, el conjunto de competencias desarrolladas en las doce semanas previas: el análisis y diseño de algoritmos según la metodología de Pólya, la construcción de programas modulares y documentados, el uso de estructuras de datos y de archivos, la recursividad, el análisis de eficiencia y el uso reflexivo de herramientas de IA generativa.

El proyecto no introduce contenidos nuevos: su valor está en la **integración**. Resolver un problema realista exige decidir qué técnica aplicar a cada parte, articular las partes en una solución coherente, repartir el trabajo en un equipo y sostener el proceso a lo largo de varias semanas. Esa experiencia de integración —más cercana a la práctica profesional que cualquier ejercicio aislado— es el objetivo central de esta etapa.

El trabajo es **colaborativo**: se desarrolla en equipos de 2 o 3 integrantes, con seguimiento individual de las contribuciones a través del repositorio compartido. El proyecto se evalúa de manera integral —el producto, el proceso y la defensa oral— con la rúbrica detallada en la sección 5 de este documento.

Relación con el curso — *Este proyecto continúa directamente la Actividad 6 de la clase práctica de la Semana 12, donde cada equipo se conformó y elaboró un plan preliminar. El presente documento formaliza la consigna, ofrece el catálogo de dominios y detalla la rúbrica cuyas cinco dimensiones se anticiparon en aquella actividad.*

2. Organización del trabajo

2.1. Equipos

Los equipos son de 2 o 3 integrantes y se mantienen estables durante las cuatro semanas. Cada equipo trabaja sobre un repositorio compartido (por ejemplo, en GitHub) donde versiona la totalidad del proyecto: código, documentación e informe. El historial de commits es parte de la evidencia del proceso —permite al docente apreciar la evolución del trabajo y la contribución de cada integrante— y se valora como tal, no como una métrica de cantidad de commits.

2.2. Elección del problema: catálogo o problema propio

Cada equipo desarrolla **un** problema, elegido por una de estas dos vías:

- (a) Dominio del catálogo.** El equipo elige uno de los cinco dominios caracterizados en la sección 4. Cada caracterización fija un núcleo obligatorio de funcionalidades, un conjunto de extensiones opcionales entre las que el equipo selecciona, y una cota explícita de alcance. Es la vía recomendada y la más directa.

- (b) Problema propio.** El equipo que prefiera abordar un problema no incluido en el catálogo puede hacerlo, con una condición: debe presentar una caracterización propia con la misma estructura que las del catálogo (núcleo obligatorio, extensiones, contenidos del curso involucrados, casos de análisis de referencia y cota de alcance) y validarla con el docente en la primera clase de la Semana 13. La elaboración de esa caracterización es, en sí misma, un ejercicio de diseño valioso, pero exige reconocer con criterio qué hace que un problema sea adecuado para el proyecto.

Validación obligatoria en la Semana 13 — Sea cual fuere la vía elegida, el problema y su alcance se validan con el docente en la clase teórica de la Semana 13. Para un dominio del catálogo, la validación se centra en el recorte de extensiones y en los casos de análisis. Para un problema propio, la validación abarca además la pertinencia y el alcance del dominio. Ningún equipo avanza a la fase de diseño sin esta validación.

2.3. Qué caracteriza a un problema adecuado

Tanto para elegir dentro del catálogo como para proponer un problema propio, conviene tener presentes los criterios que distinguen a un buen problema de proyecto. Son los mismos que se introdujeron en la Actividad 6 de la Semana 12:

Criterio	Qué significa
Tractabilidad	Es resoluble en cuatro semanas con las herramientas del curso: listas, tuplas, diccionarios, conjuntos, archivos de texto y binarios, funciones, recursión. No requiere tecnologías ajenas al curso.
Riqueza	Demanda integrar al menos tres o cuatro contenidos distintos del curso, no uno solo. Un problema que se resuelve con una única técnica no sirve como proyecto integrador.
Modularidad	Se descompone con naturalidad en módulos o funciones asignables a distintos integrantes del equipo.
Verificabilidad	Admite casos de análisis claros: para cada entrada de prueba se puede decir con precisión cuál es la salida esperada.
Alcance acotado	Está delimitado de forma explícita. Se sabe qué incluye y, sobre todo, qué no incluye.

3. Cronograma: las cuatro fases

El proyecto se organiza en cuatro fases, una por semana, que siguen las fases de Pólya a escala de proyecto. Cada fase tiene una entrega asociada. El detalle de actividades de cada semana figura en la planificación del curso; aquí se resume la orientación y los productos esperados.

Fase 1 — Semana 13: Análisis y diseño del algoritmo

Validación del problema con el docente. Construcción de los casos de análisis exhaustivos —escenarios normales, límite y extremos—. Diseño del algoritmo en pseudocódigo o literate

programming. Elección justificada de las estructuras de datos y los algoritmos. Esquema preliminar de funciones y módulos, con revisión entre pares.

- Entrega de la Fase 1: el diseño completo del proyecto —pseudocódigo o literate programming documentado, casos de análisis y descomposición en módulos—, más un avance de la codificación del módulo principal y el registro de interacciones con IA generativa.

Fase 2 — Semana 14: Codificación e integración

Codificación de los módulos siguiendo las buenas prácticas del curso. Manejo de errores con try/except. Integración de los módulos y resolución de los conflictos de integración. Pruebas con los casos de análisis definidos en la Fase 1. Documentación interna completa: docstrings con precondiciones y postcondiciones, comentarios descriptivos.

- Entrega de la Fase 2: el código integrado y documentado, la ejecución de todos los casos de análisis con el registro de sus resultados, y el avance del análisis del uso de IA generativa.

Fase 3 — Semana 15: Evaluación, optimización y preparación de la defensa

Análisis de la eficiencia del proyecto: complejidad temporal y espacial. Identificación de cuellos de botella. Al menos una mejora algorítmica documentada y justificada. Revisión final del código —consistencia de estilo, documentación, manejo de errores—. Preparación de la defensa oral y ensayo de presentaciones preliminares con devolución entre pares y del docente.

- Entrega de la Fase 3: el código final optimizado y el informe escrito completo (descripción del problema, diseño, decisiones, análisis de complejidad y análisis crítico del uso de IA), más la autoevaluación y coevaluación de la contribución de cada integrante.

Fase 4 — Semana 16: Presentación y defensa oral

Presentación final de cada equipo (15 a 20 minutos): exposición del problema, el diseño, la implementación y los resultados. Defensa oral: respuesta a las preguntas del docente y de los pares sobre las decisiones algorítmicas, estructurales y de diseño. La evaluación combina la rúbrica de resolución de problemas con los criterios del proyecto final.

- Entrega final: el repositorio cerrado —código final, README, historial de commits— y la presentación utilizada en la defensa.

Entregas y seguimiento — Cada fase cierra con una entrega en el repositorio compartido. El seguimiento es semanal: el docente revisa el avance de cada equipo en la clase práctica y en las consultas. Las entregas no son compartimentos estancos: el código entregado en la Fase 2 se optimiza en la Fase 3, y el diseño de la Fase 1 puede ajustarse si la codificación revela un problema. Lo que se evalúa es el proceso completo, no cada entrega por separado.

4. Catálogo de dominios

El catálogo ofrece cinco dominios-problema. Cada uno está caracterizado con la misma estructura: un **núcleo obligatorio** de funcionalidades que todo proyecto sobre ese dominio debe implementar; un conjunto de **extensiones opcionales** entre las que el equipo elige; los **contenidos del curso** que el dominio demanda; una muestra de **casos de análisis** que ilustra el nivel de exhaustividad esperado; y una **cota de alcance** que delimita explícitamente qué queda fuera del proyecto.

El núcleo obligatorio fija el piso de exigencia: garantiza que el proyecto integre las competencias del curso y que todos los equipos compartan una base comparable. Las extensiones dan margen a la creatividad y a los equipos que quieran un desafío mayor. **Para aprobar, un proyecto debe completar el núcleo obligatorio e implementar al menos una extensión.** El recorte concreto de extensiones se acuerda con el docente en la validación de la Semana 13.

Cómo leer una caracterización — La caracterización fija el QUÉ (qué debe hacer el sistema) y deja al equipo el CÓMO (qué algoritmo y qué estructura resuelven cada parte, cómo se descompone en módulos, cómo se reparte el trabajo). Todas las decisiones de diseño siguen siendo del equipo: el catálogo acota el problema, no lo resuelve.

Dominio A — Sistema de gestión de una biblioteca

Un sistema que administra el catálogo de ejemplares de una biblioteca y el registro de préstamos a los socios. La información de los ejemplares y de los socios se conserva de forma persistente entre ejecuciones.

Núcleo obligatorio

- Persistencia de los ejemplares en un archivo binario de registros de longitud fija (código, título, autor, año, estado), con alta de ejemplares y lectura por acceso directo.
- Persistencia de los socios en un archivo, con alta y consulta.
- Registro de préstamos y devoluciones: un préstamo asocia un socio con un ejemplar y una fecha; la devolución actualiza el estado del ejemplar.
- Índice en memoria (diccionario) que permita localizar un ejemplar por su código en tiempo $O(1)$.
- Listado del catálogo ordenado por título o por autor, mediante un algoritmo de ordenamiento implementado por el equipo.
- Menú de consola que articule todas las operaciones.

Extensiones opcionales (elegir al menos una)

- Búsqueda de ejemplares por coincidencia parcial del título (subcadena).
- Detección de préstamos vencidos a partir de la fecha de préstamo y un plazo configurable.
- Reporte de los socios con más préstamos históricos, ordenado de mayor a menor.

- Cola de reservas para un ejemplar prestado: el primero en reservar es el primero en ser notificado al devolverse (TAD cola).
- Estadísticas del catálogo: cantidad de ejemplares por autor, por año o por estado.

Contenidos del curso involucrados

Archivos binarios y módulo struct (S8); diccionarios como índice (S10); algoritmos de ordenamiento (S6, S12); búsqueda (S5); TAD cola para la extensión de reservas (S10); modularización y manejo de archivos con with (S4, S7).

Casos de análisis de referencia

- Caso normal: registrar un préstamo de un ejemplar disponible; verificar que su estado pasa a «prestado».
- Caso límite: intentar prestar un ejemplar ya prestado; el sistema debe rechazar la operación con un mensaje claro.
- Caso límite: buscar un ejemplar por un código inexistente; el sistema debe informarlo sin interrumpirse.
- Caso extremo: listar el catálogo cuando está vacío; el sistema debe producir un listado vacío, no un error.

Cota de alcance

El proyecto no incluye interfaz gráfica (la interacción es por consola), ni base de datos (la persistencia es mediante archivos), ni acceso por red ni gestión de usuarios con contraseña.

Dominio B — Gestor de inventario de un depósito

Un sistema que controla el stock de productos de un depósito: registra entradas y salidas de mercadería, mantiene actualizadas las existencias y alerta sobre los productos que requieren reposición.

Núcleo obligatorio

- Persistencia de los productos en un archivo binario de registros de longitud fija (código, descripción, cantidad en stock, stock mínimo, precio unitario).
- Registro de movimientos de entrada y de salida de mercadería, cada uno con su efecto sobre el stock del producto correspondiente.
- Índice en memoria (diccionario) para localizar un producto por su código en tiempo $O(1)$.
- Detección de los productos cuyo stock cayó por debajo de su stock mínimo (productos a reponer).
- Listado del inventario ordenado por descripción o por cantidad en stock, con un algoritmo de ordenamiento implementado por el equipo.
- Menú de consola que articule todas las operaciones.

Extensiones opcionales (elegir al menos una)

- Valorización del inventario: cálculo del valor total del stock (suma de cantidad por precio de cada producto).

- Historial de movimientos persistente, con consulta de los movimientos de un producto dado.
- Reporte de los productos de mayor rotación a partir del historial de movimientos.
- Registro de una salida que excede el stock disponible: el sistema debe rechazarla o registrar el faltante, según una política definida y documentada por el equipo.
- Estadísticas del depósito: cantidad de productos por rango de precio o por estado de stock.

Contenidos del curso involucrados

Archivos binarios y módulo struct (S8); diccionarios como índice (S10); algoritmos de ordenamiento (S6, S12); búsqueda (S5); patrón de recorrido con acumulador para la valorización (S3, S11); modularización (S4).

Casos de análisis de referencia

- Caso normal: registrar una entrada de mercadería; verificar que el stock del producto aumenta en la cantidad correspondiente.
- Caso límite: registrar una salida que deja el stock exactamente en el mínimo; el producto aún no debe aparecer como «a reponer».
- Caso límite: registrar una salida que deja el stock por debajo del mínimo; el producto debe aparecer como «a reponer».
- Caso extremo: registrar una salida mayor que el stock disponible; el sistema debe aplicar la política definida sin interrumpirse.

Cota de alcance

El proyecto no incluye interfaz gráfica (la interacción es por consola), ni base de datos (la persistencia es mediante archivos), ni acceso por red ni integración con sistemas de facturación.

Dominio C — Planificador de horarios de cursada

Un sistema que asigna materias a franjas horarias respetando un conjunto de restricciones, de modo que no se produzcan superposiciones. Es el dominio con mayor componente de búsqueda combinatoria del catálogo.

Núcleo obligatorio

- Representación de las materias, las franjas horarias y las aulas mediante estructuras de datos adecuadas, justificando la elección.
- Carga de los datos de entrada (materias a planificar, franjas disponibles, restricciones) desde un archivo de texto.
- Asignación de cada materia a una franja y un aula mediante backtracking, respetando las restricciones: una franja-aula no puede tener dos materias, y una materia no puede asignarse a una franja incompatible.
- Detección de la situación sin solución: si las restricciones son incompatibles, el sistema debe informarlo en lugar de quedar en un bucle.

- Presentación del horario resultante en una grilla legible por consola.
- Menú de consola que articule la carga, la planificación y la presentación.

Extensiones opcionales (elegir al menos una)

- Restricciones de disponibilidad por docente: un docente solo puede dictar en ciertas franjas.
- Conteo de todas las planificaciones válidas posibles, además de encontrar una.
- Preferencias blandas: entre dos planificaciones válidas, elegir la que mejor satisface un criterio de preferencia (por ejemplo, minimizar las franjas libres entre materias).
- Persistencia de la planificación obtenida en un archivo, y su recarga posterior.
- Medición de la cantidad de nodos del árbol de búsqueda explorados, para evidenciar el efecto de la poda.

Contenidos del curso involucrados

Backtracking (S12); estructuras de datos para el estado parcial (S10); archivos de texto (S7); recursividad y árbol de búsqueda (S9, S12); análisis de crecimiento combinatorio (S12); modularización (S4).

Casos de análisis de referencia

- Caso normal: un conjunto de materias y franjas con solución holgada; el sistema debe encontrar una asignación válida.
- Caso límite: un conjunto con solución única (las restricciones determinan una sola planificación posible).
- Caso límite: un conjunto sobre-restringido sin solución; el sistema debe informarlo tras explorar el espacio.
- Caso extremo: una sola materia y una sola franja compatible; la planificación debe ser inmediata y correcta.

Cota de alcance

El proyecto no incluye interfaz gráfica (la interacción es por consola), ni optimización con técnicas ajenas al curso. La búsqueda se resuelve con backtracking; no se requieren algoritmos de programación lineal ni bibliotecas de resolución de restricciones.

Dominio D — Analizador de archivos de registro (logs)

Un sistema que procesa un archivo de registros de actividad —un log, como el que genera un servidor o una aplicación—, extrae información estructurada de cada línea y produce estadísticas y reportes ordenados. Es el dominio con mayor componente de procesamiento de secuencias y cadenas.

Núcleo obligatorio

- Lectura de un archivo de texto de log, línea por línea, con el context manager with.
- Análisis (parsing) de cada línea para extraer sus campos (por ejemplo: fecha y hora, nivel de severidad, origen, mensaje), tratando la línea como una secuencia de caracteres.

- Almacenamiento de los registros analizados en estructuras de datos adecuadas para su posterior consulta.
- Cómputo de estadísticas: cantidad de registros por nivel de severidad, por origen o por franja horaria, usando diccionarios como contadores.
- Reporte de los resultados ordenado según un criterio (por ejemplo, los orígenes con más errores, de mayor a menor), con un algoritmo de ordenamiento implementado por el equipo.
- Menú de consola que articule la carga, el análisis y los reportes.

Extensiones opcionales (elegir al menos una)

- Filtrado de registros por un rango de fechas o por nivel de severidad.
- Detección de ráfagas: períodos en los que la cantidad de errores por unidad de tiempo supera un umbral.
- Manejo robusto de líneas mal formadas: el analizador debe descartarlas o registrarlas como inválidas sin interrumpirse.
- Exportación del reporte a un archivo de texto con un formato tabular legible.
- Búsqueda de registros cuyo mensaje contenga una subcadena dada.

Contenidos del curso involucrados

Archivos de texto (S7); procesamiento de cadenas como secuencias (S3); diccionarios como contadores (S10); algoritmos de ordenamiento (S6, S12); patrón de recorrido con acumulador (S3, S11); manejo de errores para las líneas mal formadas (S12); modularización (S4).

Casos de análisis de referencia

- Caso normal: un log con varias decenas de líneas bien formadas; el sistema debe producir las estadísticas correctas.
- Caso límite: un log con una sola línea; las estadísticas deben ser correctas y el reporte no debe fallar.
- Caso límite: un archivo de log vacío; el sistema debe producir estadísticas vacías sin error.
- Caso extremo: un log con líneas mal formadas intercaladas; el analizador debe procesarlas según la política definida.

Cota de alcance

El proyecto no incluye interfaz gráfica (la interacción es por consola), ni análisis en tiempo real (se procesa un archivo ya existente, no un flujo en vivo), ni expresiones regulares (el parsing se resuelve con operaciones de cadena del curso).

Dominio E — Sistema de turnos de un centro de atención

Un sistema que gestiona los turnos de un centro de atención —un consultorio, una oficina de trámites— asignando a las personas a franjas horarias y respetando criterios de prioridad y

disponibilidad. Amplía, a escala de proyecto, el problema integrador de la clase práctica de la Semana 12.

Núcleo obligatorio

- Persistencia de las personas registradas en un archivo binario de registros de longitud fija (documento, apellido, nombre, dato de contacto, prioridad).
- Índice en memoria (diccionario) para localizar a una persona por su documento en tiempo $O(1)$.
- Gestión de una agenda diaria de franjas horarias: asignación de personas a franjas, respetando que cada franja reciba a lo sumo una persona.
- Asignación que respete la disponibilidad de cada persona (las franjas en las que puede asistir) mediante backtracking.
- Listado de las personas registradas ordenado por apellido o por prioridad, con un algoritmo de ordenamiento estable implementado por el equipo.
- Menú de consola que articule todas las operaciones.

Extensiones opcionales (elegir al menos una)

- Atención por orden de prioridad dentro de una franja sobre-demandada, usando una cola con prioridad (TAD).
- Persistencia de la agenda asignada en un archivo y su recarga posterior.
- Reasignación: ante la cancelación de un turno, ofrecer la franja liberada a otra persona compatible.
- Reporte de ocupación: porcentaje de franjas asignadas, franjas libres, personas sin turno.
- Detección de la agenda sin solución (más personas que franjas compatibles) e informe de las personas que no pudieron ubicarse.

Contenidos del curso involucrados

Archivos binarios y módulo struct (S8); diccionarios como índice (S10); backtracking para la asignación (S12); algoritmos de ordenamiento estables (S6, S12); TAD cola con prioridad para la extensión (S10); modularización (S4).

Casos de análisis de referencia

- Caso normal: asignar una agenda donde todas las personas tienen franjas compatibles holgadas; la asignación debe ser válida.
- Caso límite: una agenda con solución única por las restricciones de disponibilidad.
- Caso límite: una agenda sobre-restringida sin solución; el sistema debe informarlo.
- Caso extremo: ordenar el listado por prioridad cuando varias personas comparten la misma prioridad; el orden relativo por apellido debe preservarse (estabilidad).

Cota de alcance

El proyecto no incluye interfaz gráfica (la interacción es por consola), ni base de datos (la persistencia es mediante archivos), ni acceso por red ni notificaciones automáticas reales.

5. Rúbrica de evaluación

El proyecto se evalúa de manera integral según cinco dimensiones. Cada dimensión se valora en cuatro niveles de logro. La evaluación considera el producto final, el proceso evidenciado en el repositorio y las entregas de cada fase, y la defensa oral. La rúbrica se conoce desde el inicio del proyecto para que cada equipo pueda orientar su trabajo.

5.1. Dimensión 1 — Pensamiento algorítmico

Calidad del análisis del problema, pertinencia de las estructuras de datos y los algoritmos elegidos, y comprensión de la eficiencia de la solución.

Destacado	El análisis identifica con precisión los casos normales, límite y extremos. Las estructuras y los algoritmos elegidos son los más adecuados y la elección está justificada. El análisis de complejidad es correcto y se usa para fundamentar decisiones.
Satisfactorio	El análisis cubre los casos principales. Las estructuras y los algoritmos son adecuados, con una justificación razonable. El análisis de complejidad es esencialmente correcto.
En desarrollo	El análisis es incompleto u omite casos límite relevantes. Las elecciones de estructura o algoritmo son funcionales pero no están justificadas o no son las más adecuadas. El análisis de complejidad es impreciso.
Inicial	El análisis es insuficiente. Las estructuras o los algoritmos elegidos no son apropiados para el problema. No hay análisis de complejidad o es incorrecto.

5.2. Dimensión 2 — Modularización y reutilización

Descomposición del problema en módulos y funciones con responsabilidades claras, interfaces bien definidas y ausencia de duplicación de código.

Destacado	La descomposición es clara y natural: cada función tiene una responsabilidad única y bien delimitada. Las interfaces están bien definidas. No hay duplicación. La lógica del problema está separada de la entrada/salida.
Satisfactorio	La descomposición es adecuada: la mayoría de las funciones tienen una responsabilidad clara. Las interfaces son razonables. La duplicación es mínima.
En desarrollo	La descomposición es irregular: algunas funciones concentran demasiada responsabilidad o hay duplicación de código apreciable. La separación entre lógica y E/S es parcial.
Inicial	El programa está escasamente modularizado: funciones extensas con múltiples responsabilidades, duplicación notoria, lógica y E/S entremezcladas.

5.3. Dimensión 3 — Documentación interna

Calidad de las docstrings con precondiciones y postcondiciones, pertinencia de los comentarios y documentación de los casos de análisis.

Destacado	Toda función tiene docstring con precondición y postcondición; los efectos secundarios están declarados. Los comentarios explican el porqué de las decisiones, no parafrasean el código. Los casos de análisis están documentados con entrada y salida esperada.
Satisfactorio	La mayoría de las funciones tienen docstring con contrato. Los comentarios son en general pertinentes. Los casos de análisis están documentados.
En desarrollo	Las docstrings son irregulares o incompletas (faltan precondiciones o postcondiciones). Algunos comentarios solo parafrasean el código. La documentación de los casos de análisis es parcial.
Inicial	Las docstrings están ausentes o son meramente nominales. Los comentarios son escasos o redundantes. Los casos de análisis no están documentados.

5.4. Dimensión 4 — Calidad del producto

Corrección del programa en los casos normales y límite, completitud respecto del núcleo y las extensiones comprometidas, y manejo de errores.

Destacado	El programa resuelve correctamente todos los casos de análisis, incluidos los límite y extremos. El núcleo obligatorio y las extensiones comprometidas están completos. El manejo de errores es robusto: ninguna entrada inesperada interrumpe el programa.
Satisfactorio	El programa resuelve los casos normales y la mayoría de los límite. El núcleo está completo y al menos una extensión funciona. El manejo de errores cubre las situaciones más frecuentes.
En desarrollo	El programa resuelve los casos normales pero falla en varios casos límite. El núcleo está casi completo. El manejo de errores es parcial: algunas entradas inesperadas interrumpen el programa.
Inicial	El programa falla en casos normales o el núcleo obligatorio está incompleto. No hay manejo de errores apreciable.

5.5. Dimensión 5 — Proceso, ética y comunicación

Evidencia del proceso de trabajo en el repositorio, contribución de cada integrante, registro y análisis crítico del uso de IA generativa, y calidad de la defensa oral.

Destacado	El historial de versionado evidencia un trabajo sostenido y la contribución de todos los integrantes. El registro del uso de IA es completo y el análisis crítico es genuino. La defensa oral es clara y cada integrante fundamenta sólidamente las decisiones del proyecto.
------------------	--

Satisfactorio	El repositorio evidencia el proceso y la contribución de los integrantes. El uso de IA está registrado y analizado. La defensa oral es adecuada y el equipo responde a las preguntas.
En desarrollo	El historial de versionado es irregular o la contribución de los integrantes es desigual. El registro del uso de IA es superficial. La defensa oral es despareja: no todos los integrantes pueden fundamentar las decisiones.
Inicial	El repositorio no evidencia el proceso o la contribución es muy desigual. No hay registro del uso de IA o es meramente nominal. La defensa oral no logra fundamentar las decisiones del proyecto.

Sobre la evaluación individual dentro del equipo — *El proyecto es colaborativo, pero la evaluación contempla la contribución individual. El historial del repositorio, la autoevaluación y la coevaluación entre integrantes, y la defensa oral —donde cada integrante debe poder fundamentar las decisiones del proyecto— permiten al docente apreciar el aporte de cada uno. Un integrante que no pueda dar cuenta del trabajo no se beneficia de la calificación del equipo.*

6. Uso de IA generativa en el proyecto

El uso de herramientas de IA generativa durante el proyecto está **permitido y, a la vez, sujeto a registro y análisis crítico**. El curso no prohíbe estas herramientas —forman parte del ejercicio profesional actual— pero exige usarlas de un modo que favorezca el aprendizaje y no lo sustituya.

Cada equipo mantiene, a lo largo de las cuatro semanas, un registro de sus interacciones significativas con herramientas de IA. Para cada interacción relevante, el registro consigna:

- el prompt utilizado;
- la respuesta obtenida (o un resumen fiel de ella);
- qué se tomó, qué se descartó y por qué;
- qué verificación se hizo sobre el código o la información obtenida.

El informe final incluye un apartado de **análisis crítico** del uso de IA: qué aportó, qué riesgos se identificaron, qué errores o imprecisiones se detectaron en lo generado, y qué aprendió el equipo sobre el uso de estas herramientas. La responsabilidad sobre el código entregado es siempre del equipo: cada integrante debe poder explicar y fundamentar cualquier parte del proyecto, haya sido escrita con ayuda de IA o no.

Honestidad académica — *Entregar como propio un trabajo que el equipo no comprende ni puede explicar —provenga de una IA, de otro equipo o de cualquier otra fuente— es una falta de honestidad académica. El registro del uso de IA y la defensa oral existen, entre otras razones, para que la evaluación refleje el trabajo y la comprensión genuinos del equipo.*

7. Síntesis de entregables

La tabla resume los productos esperados al cierre de cada fase. Todas las entregas se realizan en el repositorio compartido del equipo.

Fase / Semana	Entregable
Fase 1 — Semana 13	Diseño completo: pseudocódigo o literate programming, casos de análisis, descomposición en módulos. Avance del módulo principal. Inicio del registro de uso de IA.
Fase 2 — Semana 14	Código integrado y documentado. Ejecución de todos los casos de análisis con sus resultados. Avance del análisis de uso de IA.
Fase 3 — Semana 15	Código final optimizado (con al menos una mejora documentada). Informe escrito completo. Autoevaluación y coevaluación.
Fase 4 — Semana 16	Repositorio cerrado: código final, README, historial de commits. Presentación de la defensa oral.

Contenido del informe escrito

El informe escrito, que se entrega al cierre de la Fase 3, integra el trabajo de las cuatro fases. Su estructura esperada es:

- Descripción del problema y del alcance abordado (dominio, núcleo y extensiones implementadas).
- Diseño de la solución: descomposición en módulos, estructuras de datos y algoritmos elegidos, con su justificación.
- Casos de análisis: la batería completa, con la entrada y la salida esperada de cada uno, y el resultado obtenido.
- Análisis de complejidad temporal y espacial de las partes significativas del proyecto.
- La mejora algorítmica de la Fase 3: qué se optimizó, por qué y con qué efecto.
- Análisis crítico del uso de IA generativa.
- Reflexión final del equipo sobre el proceso de aprendizaje.

Una recomendación de cierre — El proyecto premia el trabajo sostenido por sobre el esfuerzo final concentrado. Un equipo que llega a la Semana 14 con un diseño sólido —algoritmo pseudocodificado, casos de análisis definidos, módulos esquematizados— codifica con calma lo que un equipo sin diseño previo codifica y recodifica a las apuradas. Dedicar tiempo al análisis y al diseño en la Fase 1 no es tiempo perdido: es la mejor inversión del proyecto.